Universidad Don Bosco, El Salvador

Desarrollo de Software para Móviles DSM941

FORO #2

Integrantes del equipo:

Nombres	Numero de Carnet
Alejandro Benjamín Rivera Ochoa	RO243154
Mayron Steve Lopez Girón	LG243153
Andrea Paola Montenegro Paz	MP101106
Marlon Eduardo Montenegro Paz	MP243207
Miguel Vallejos	VL131638

Enlace GITHUB:

https://github.com/MarlonMontenegro/personal-expenses-app.git

Enlace video:

Personal Expenses app.mp4

Índice

Introducción 3
Investigación sobre Autenticación 4
¿Qué es la autenticación en aplicaciones móviles? 4
Firebase Authentication 4
Flujo de Autenticación Implementado 4
Registro de Usuarios 5
Desarrollo Técnico Paso a Paso 5
1. Configuración del Entorno de Desarrollo 5
1.1 Herramientas Utilizadas 5
1.2 Configuración de Firebase 5
2. Arquitectura del Proyecto (MVC) 6
2.1 Modelo — model/Expense.kt 6
2.2 Controladores — controller/ 6
2.3 Vistas — views/ 7
3. Estructura de Base de Datos en Firestore 7
4. Flujo de Navegación 8
5. Validaciones del Lado del Cliente 8
6. Interfaz de Usuario — Diseño Visual 9
7. Buenas Prácticas de Ingeniería de Software Aplicadas 9
Capturas de Pantalla del Proyecto en Ejecución 10
Pantalla de Inicio de Sesión (LoginActivity) 10
Pantalla de Registro (RegisterActivity) 11
Dashboard Principal (DashboardActivity) 11
Agregar Gasto (AddExpenseActivity) 12
Historial de Gastos (HistoryActivity) 12
Firebase Console — Usuarios Registrados 13
Firebase Console — Estructura de Datos en Firestore 13
Conclusiones Personales del Grupo 14
Referencias 15

Introducción

El manejo de las finanzas personales es una habilidad fundamental en la vida cotidiana. Sin embargo, muchas personas carecen de herramientas accesibles y fáciles de usar que les permitan llevar un registro ordenado de sus gastos. La digitalización de este proceso mediante aplicaciones móviles representa una solución práctica y eficiente para este problema.

Personal Expenses App surge como respuesta a esta necesidad: una aplicación Android desarrollada en Kotlin que permite a los usuarios registrar, visualizar y analizar sus gastos personales de forma segura, desde cualquier dispositivo y en cualquier momento. Al utilizar la infraestructura de Firebase de Google, la aplicación garantiza que los datos estén siempre disponibles en la nube, sin depender del almacenamiento local del dispositivo.

Este documento presenta la documentación técnica completa del proyecto, incluyendo su arquitectura, las tecnologías utilizadas, el flujo de navegación entre pantallas, el modelo de datos empleado y las decisiones de diseño tomadas durante el desarrollo. El objetivo es proporcionar una visión integral del sistema que permita comprender cómo funciona cada componente y cómo se integran entre sí.

El proyecto fue desarrollado como ejercicio académico en la Universidad Dr. José Matías Delgado, aplicando conceptos fundamentales de desarrollo Android moderno: el patrón de arquitectura MVC, autenticación de usuarios, bases de datos en la nube y validación de formularios del lado del cliente.

Investigación sobre Autenticación

¿Qué es la autenticación en aplicaciones móviles?

La autenticación es el proceso mediante el cual un sistema verifica la identidad de un usuario antes de otorgarle acceso a recursos o funcionalidades protegidas. En el contexto de aplicaciones móviles, este proceso resulta esencial para garantizar que cada usuario acceda únicamente a su propia información y que los datos sensibles se mantengan seguros (Sommerville, 2016).

Existen diferentes métodos de autenticación: usuario y contraseña, autenticación biométrica (huella digital, reconocimiento facial), tokens de sesión y autenticación de dos factores (2FA). Para este proyecto se seleccionó el método clásico de correo electrónico y contraseña, implementado mediante Firebase Authentication, por su simplicidad de integración y confiabilidad en producción.

Firebase Authentication

Firebase Authentication es un servicio de Google que proporciona una infraestructura completa para la gestión de identidades de usuarios. Elimina la necesidad de construir y mantener un servidor de autenticación propio, delegando en Google la seguridad del almacenamiento de credenciales y la gestión de sesiones (Firebase, 2024).

Entre sus características más relevantes se encuentran:

- Soporte para múltiples proveedores de identidad: correo/contraseña, Google, Facebook, GitHub, entre otros.
- Tokens de sesión seguros basados en JSON Web Tokens (JWT) con expiración automática.
- SDK nativo para Android y iOS con integración directa en Kotlin y Swift.
- Panel de administración en Firebase Console para gestionar usuarios registrados.
- Reglas de seguridad integrables con Cloud Firestore para control de acceso a nivel de datos.

Flujo de Autenticación Implementado

En Personal Expenses App, el flujo de autenticación sigue los siguientes pasos:

1. El usuario ingresa su correo electrónico y contraseña en LoginActivity.
2. El cliente valida localmente el formato del correo (Patterns.EMAIL_ADDRESS) y que la contraseña no esté vacía.
3. Se invoca auth.signInWithEmailAndPassword(email, password) de la SDK de Firebase.
4. Firebase verifica las credenciales contra su base de datos segura en la nube.
5. Si son válidas, retorna un FirebaseUser con el UID único del usuario.
6. La aplicación redirige al usuario a DashboardActivity usando finish() para evitar retroceso al login.

Registro de Usuarios

El proceso de registro es análogo al inicio de sesión. El usuario proporciona correo y contraseña (mínimo 6 caracteres, requisito de Firebase), y se invoca auth.createUserWithEmailAndPassword(). Firebase almacena las credenciales de forma cifrada y retorna el FirebaseUser recién creado, quien es redirigido automáticamente al Dashboard.

Firebase Authentication garantiza que los correos sean únicos por aplicación: si un usuario intenta registrarse con un correo ya existente, el servicio retorna un error descriptivo que la aplicación comunica al usuario mediante un Toast.

Desarrollo Técnico Paso a Paso

1. Configuración del Entorno de Desarrollo

1.1 Herramientas Utilizadas

Tecnología / Librería	Versión	Propósito
Kotlin	JVM 11	Lenguaje principal de desarrollo
Android SDK	minSdk 24 / targetSdk 36	Plataforma objetivo
Firebase Authentication	BOM 34.1.0	Autenticación de usuarios
Firebase Firestore	BOM 34.1.0	Base de datos NoSQL en la nube
AppCompat / Material	AndroidX	Componentes visuales y temas
Gradle (Kotlin DSL)	9.x	Sistema de construcción del proyecto

1.2 Configuración de Firebase

Para integrar Firebase en el proyecto Android se siguieron los siguientes pasos:

7. Crear un proyecto en Firebase Console (https://console.firebase.google.com).
8. Registrar la aplicación Android con el package name: com.udb.personalexpensesapp.
9. Descargar el archivo google-services.json y colocarlo en la carpeta app/ del proyecto.
10. Agregar el plugin de Google Services en los archivos build.gradle.kts.
11. Habilitar Firebase Authentication con el método de Correo/Contraseña en la consola.
12. Crear la base de datos Cloud Firestore en modo de prueba.

2. Arquitectura del Proyecto (MVC)

El proyecto implementa el patrón Modelo-Vista-Controlador (MVC), que separa las responsabilidades del sistema en tres capas bien definidas, facilitando el mantenimiento y la escalabilidad del código.

2.1 Modelo — model/Expense.kt

La clase Expense es una data class de Kotlin que representa la entidad principal del dominio. Sus campos son compatibles con la deserialización automática de Firestore mediante el constructor vacío implícito que proveen los valores por defecto:

```
data class Expense (
    val id: String = "",        // ID del documento en Firestore
    val name: String = "",      // Nombre del gasto
    val amount: Double = 0.0,   // Monto en dólares
    val category: String = "",// Categoría (ej: Comida, Transporte)
    val date: String = "",     // Fecha ingresada por el usuario
    val month: Int = 0,         // Mes calculado automáticamente
    val year: Int = 0           // Año calculado automáticamente
)
```

Los campos month y year se extraen automáticamente mediante Calendar.getInstance() al momento de guardar el gasto, lo que permite filtrar eficientemente los gastos del mes actual para el cálculo del total mensual.

2.2 Controladores — controller/

Los controladores encapsulan toda la lógica de negocio y la comunicación con Firebase. Las vistas nunca acceden directamente a los servicios de Firebase; toda interacción pasa obligatoriamente a través de los controladores.

AuthController.kt

Maneja el ciclo completo de autenticación de usuarios:

Método	Descripción
registerWithEmail(email, password, onSuccess, onError)	Crea un nuevo usuario en Firebase Authentication.
loginWithEmail(email, password, onSuccess, onError)	Autentica a un usuario existente.
logout()	Cierra la sesión del usuario actual con auth.signOut().
getCurrentUser()	Retorna el FirebaseUser actualmente autenticado o null.
isUserLoggedIn()	Retorna true si hay una sesión activa, false si no.

ExpenseController.kt

Gestiona las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre la colección de gastos del usuario en Firestore:

Método	Descripción
addExpense(expense, onSuccess, onError)	Guarda un nuevo gasto en la subcolección expenses del usuario.
getExpenses(onSuccess, onError)	Obtiene todos los gastos del usuario como lista de objetos Expense.
getMonthlyTotal(onSuccess, onError)	Filtra los gastos del mes y año actual, suma los montos y retorna el total.

Todos los métodos utilizan callbacks tipados de Kotlin (lambdas onSuccess y onError) para manejar la naturaleza asíncrona de las operaciones de red de Firebase.

2.3 Vistas — views/

Cada vista es una Activity de Android que gestiona exclusivamente la interfaz de usuario y delega toda la lógica de negocio a los controladores correspondientes.

Archivo	Descripción
LoginActivity.kt	Pantalla de inicio de sesión con validación de formulario.
RegisterActivity.kt	Pantalla de registro con validación (mín. 6 chars contraseña).
DashboardActivity.kt	Vista principal: muestra total mensual y acceso a secciones.
AddExpenseActivity.kt	Formulario para registrar un nuevo gasto con validaciones.
HistoryActivity.kt	Historial de gastos mostrado en un ListView con ArrayAdapter.

3. Estructura de Base de Datos en Firestore

Los datos se almacenan en Cloud Firestore utilizando una estructura jerárquica por usuario, lo que garantiza el aislamiento total de datos entre diferentes cuentas:

```
users (colección)
  └── {userId} (documento — UID de Firebase Auth)
        └── expenses (subcolección)
              └── {expenseId} (documento auto-generado)
                    ├── name: String
                    ├── amount: Double
                    ├── category: String
                    ├── date: String
                    ├── month: Int
                    └── year: Int
```

Cada usuario accede únicamente a sus propios datos ya que todas las consultas se realizan bajo su UID autenticado. La estructura de subcolección por usuario es el patrón recomendado por Firebase para aplicaciones multi-usuario.

4. Flujo de Navegación

El flujo de la aplicación sigue esta secuencia lógica:

13. Al iniciar la app, se carga LoginActivity (definida como LAUNCHER en el AndroidManifest).
14. El usuario puede iniciar sesión o navegar a RegisterActivity para crear una cuenta nueva.
15. Tras autenticarse exitosamente, ambas vistas redirigen a DashboardActivity usando finish(), previniendo el regreso a la pantalla de login con el botón atrás del dispositivo.
16. Desde el Dashboard, el usuario puede: agregar un gasto (AddExpenseActivity), ver el historial (HistoryActivity), o cerrar sesión (vuelve a LoginActivity).
17. El Dashboard implementa onResume() para recargar el total mensual cada vez que el usuario regresa desde otras pantallas, garantizando datos siempre actualizados.

5. Validaciones del Lado del Cliente

La aplicación implementa validaciones en los formularios antes de enviar cualquier dato a Firebase, garantizando la integridad de la información y mejorando la experiencia de usuario:

Pantalla	Campo	Validación aplicada
Login	Correo	No vacío + formato de email válido (Patterns.EMAIL_ADDRESS)
Login	Contraseña	No vacía
Registro	Correo	No vacío + formato de email válido
Registro	Contraseña	No vacía + mínimo 6 caracteres
Agregar Gasto	Nombre	No vacío
Agregar Gasto	Monto	No vacío + número decimal positivo (toDoubleOrNull + > 0)
Agregar Gasto	Categoría	No vacía
Agregar Gasto	Fecha	No vacía

6. Interfaz de Usuario — Diseño Visual

Todas las pantallas siguen un diseño consistente con LinearLayout vertical y una paleta de colores en dos tonos principales:

Color	Código HEX	Uso
Verde oscuro	#2E7D32	Acciones primarias (guardar, login, elementos de la app)
Azul oscuro	#1565C0	Acciones secundarias (registro, historial, volver)
Rojo oscuro	#B71C1C	Acciones destructivas (cerrar sesión)
Gris claro	#F5F5F5	Fondo de todas las pantallas
Gris medio	#616161	Subtítulos y textos secundarios

Los componentes de interfaz utilizados son los estándar de Android: EditText y Button para formularios de entrada, TextView para mostrar información, y ListView con ArrayAdapter para el historial de gastos.

7. Buenas Prácticas de Ingeniería de Software Aplicadas

Práctica	Implementación en el proyecto
Separación de responsabilidades (MVC)	Las vistas solo manejan UI; los controladores manejan Firebase y lógica de negocio.
Callbacks tipados de Kotlin	Uso de lambdas (onSuccess, onError) en lugar de listeners anónimos verbosos.
Encapsulamiento de Firebase	Las vistas no importan ni usan Firebase directamente; todo pasa por los controladores.
Actualización en onResume()	DashboardActivity recarga el total mensual al volver desde AddExpenseActivity.
finish() post-login	Evita que el usuario regrese a la pantalla de login usando el botón atrás del dispositivo.
Limpieza de formularios	clearFields() en AddExpenseActivity limpia el formulario tras guardar exitosamente.
Código comentado en español	Todos los métodos tienen comentarios KDoc explicando su propósito.

Capturas de Pantalla del Proyecto en Ejecución

A continuación se presentan las capturas de pantalla del proyecto ejecutándose en un dispositivo o emulador Android, mostrando el flujo completo de la aplicación.

Pantalla de Inicio de Sesión (LoginActivity)

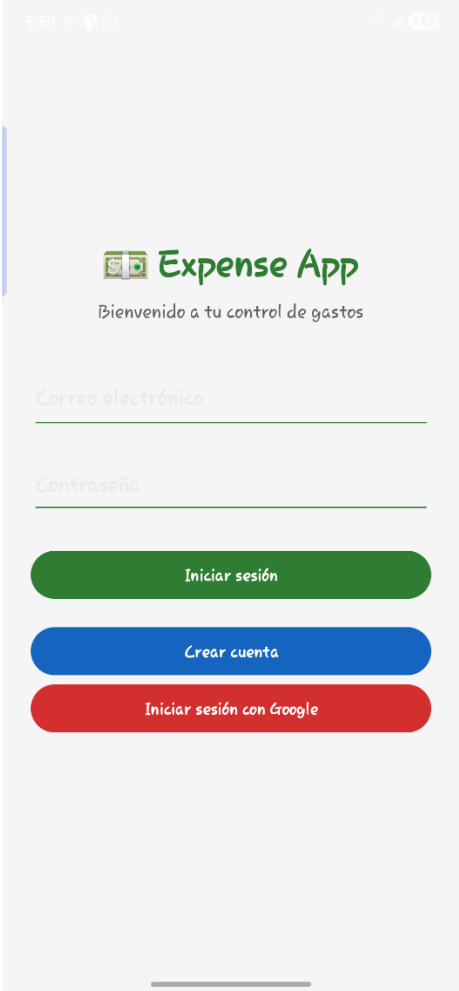

Figura 1. Pantalla de inicio de sesión de Personal Expenses App.

Pantalla de Registro (RegisterActivity)

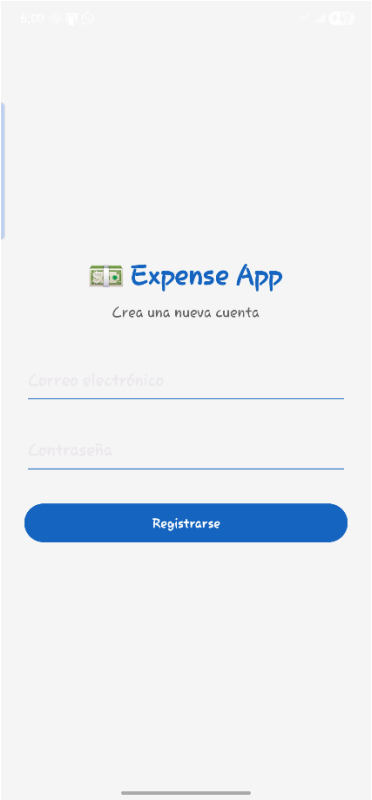

Figura 2. Pantalla de registro de nuevos usuarios.

Dashboard Principal (DashboardActivity)

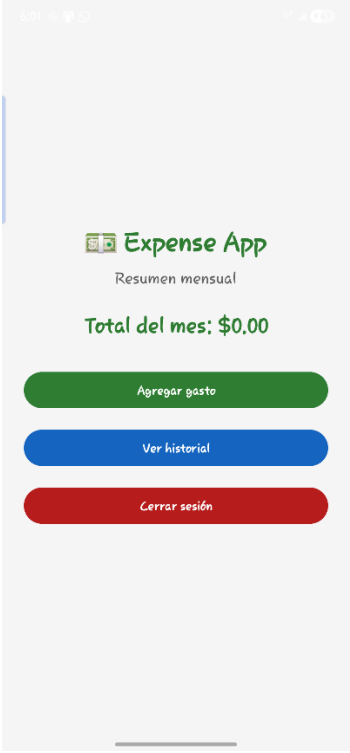

Figura 3. Dashboard principal mostrando el total mensual de gastos.

Agregar Gasto (AddExpenseActivity)

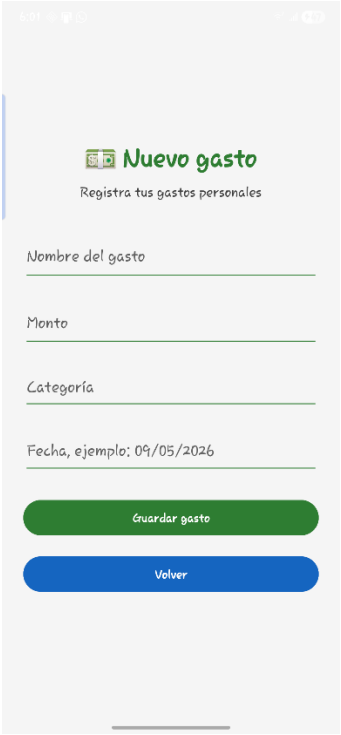

Figura 4. Formulario para el registro de un nuevo gasto.

Historial de Gastos (HistoryActivity)

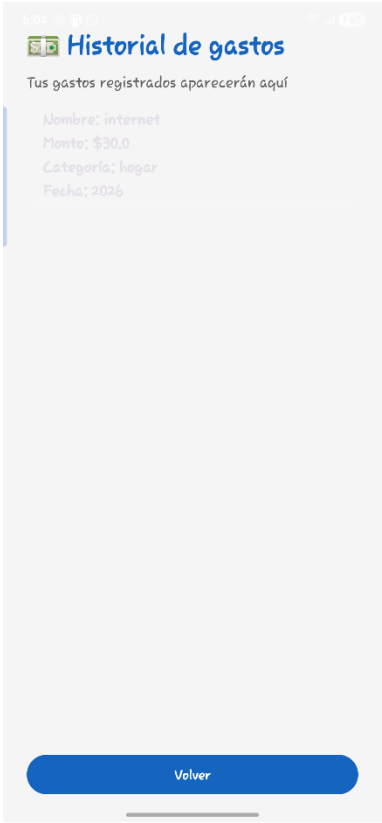

Figura 5. Historial completo de gastos registrados por el usuario.

Firebase Console — Usuarios Registrados

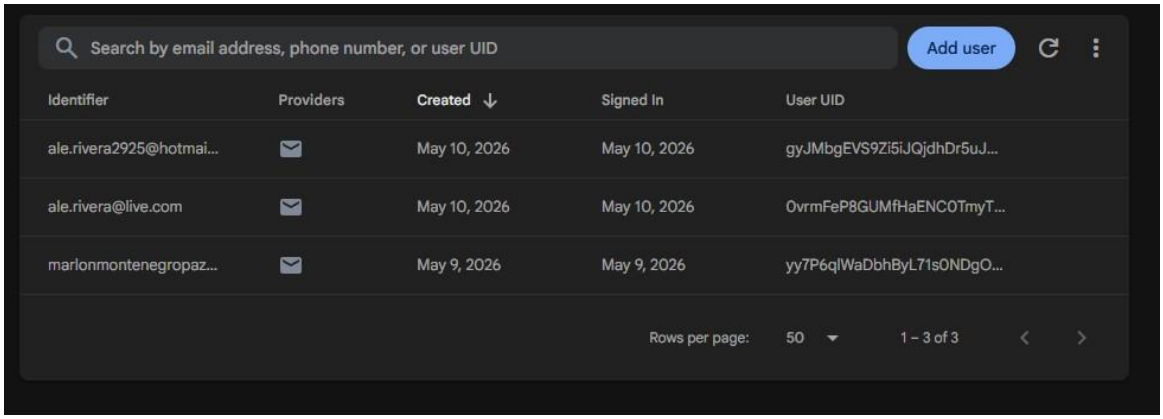

Figura 6. Firebase Console mostrando los usuarios registrados en Authentication.

Firebase Console — Estructura de Datos en Firestore

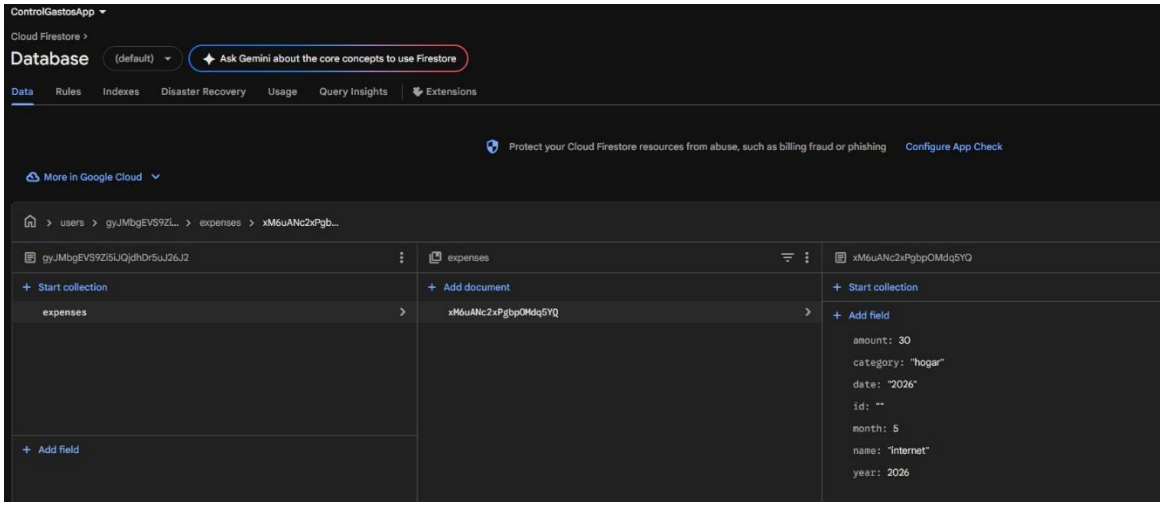

Figura 7. Estructura de datos almacenada en Cloud Firestore.

Conclusiones

El desarrollo de “Personal Expenses App” ha permitido aplicar de manera práctica un conjunto de tecnologías y conceptos fundamentales del desarrollo de software moderno para dispositivos Android. A lo largo del proyecto se demostró que es posible construir una aplicación funcional, segura y bien estructurada combinando Kotlin, el patrón MVC y los servicios de Firebase.

Desde el punto de vista técnico, la separación en capas (Modelo, Vista y Controlador) resultó ser una decisión de diseño acertada: facilitó la organización del código, redujo el acoplamiento entre componentes y hizo que la aplicación sea más fácil de mantener y extender. El uso de Firebase Authentication resolvió de forma eficiente el manejo de cuentas de usuario, mientras que Cloud Firestore proporcionó una base de datos escalable en la nube sin necesidad de configurar servidores propios.

En cuanto a la experiencia de usuario, la aplicación presenta un flujo de navegación claro y coherente: el usuario puede registrarse, iniciar sesión, registrar gastos y consultar su historial y resumen mensual en pocos pasos. Las validaciones del lado del cliente aseguran que los datos ingresados sean correctos antes de enviarlos a la nube, mejorando la confiabilidad del sistema.

Es importante reconocer que el proyecto, siendo un ejercicio académico, tiene áreas de mejora identificadas: la verificación automática de sesión activa al iniciar la app, el uso de RecyclerView en lugar de ListView para mejor rendimiento, y la implementación de reglas de seguridad en Firestore son los puntos más relevantes a atender en una versión productiva.

En conclusión, Personal Expenses App cumple con sus objetivos académicos y funcionales: es una aplicación operativa que registra y consulta gastos en la nube, con código bien organizado y documentado. Sienta las bases para un producto de finanzas personales que, con las mejoras identificadas, podría ser publicado y ser de utilidad real para sus usuarios.

Referencias

Firebase. (2024). Firebase Authentication. Google LLC. https://firebase.google.com/docs/auth

Firebase. (2024). Cloud Firestore. Google LLC. https://firebase.google.com/docs/firestore

Google. (2024). Android Developers Documentation. https://developer.android.com/docs

JetBrains. (2024). Kotlin Programming Language. https://kotlinlang.org/docs/home.html

Sommerville, I. (2016). Ingeniería del software (10.a ed.). Pearson Educación.

Freeman, E., & Robson, E. (2020). Head First Android Development: A Brain-Friendly Guide (3rd ed.). O'Reilly Media.